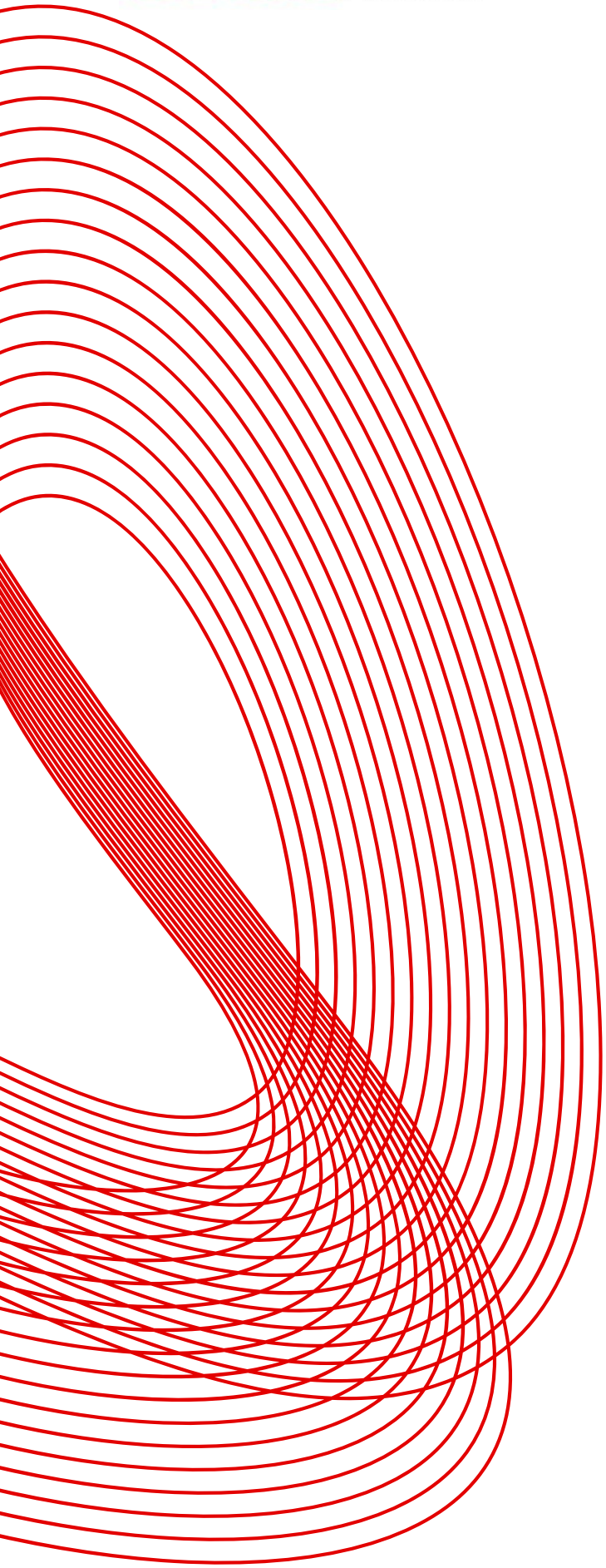
INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES STRASBOURG

S9



S-IoT Project Report :

Drone swarm

Written by :

Emilien PEYON

Noé RAMDANI

Class of 2022-2027

Contents

I Main **4**
1 UWB . . . 4
2 Data processing . . . 5
3 Communication . . . 6
4 Visualization . . . 7
A Matlab . . . 7
B Architecture . . . 7
5 Simulation . . . 8
A ArduPilot and Mission Planer . . . 8
B Architecture . . . 8
C Result . . . 9
D Simulation possibilities . . . 9
6 Outdoor Test . . . 10

List of Figures

I.1 Visualization architecture 7
I.2 Simulation architecture 9

I - Main

1 - UWB

2 - Data processing

3 - Communication

4 - Visualization

Visualization permit to obtain precise measurement and result from test in major case drone will fly outdoor at 10 meters away from us. So, vizualise permit to precisely see the attitude of the drone, the real shape positioning. It can permit to detect error, or problematic point.
To visualise, we choose the sofware Matlab. And had the objectif to be able to see the attitude, the shape position and also look at the data coming from the drone.

A - Matlab

Matlab is a general tool developped by the MathWorks. It has many functions such as mathematics calculator, simulator, and also a corrector designer. Based on add-on and toolbox, it can be usefull for many differents purpose. It also permits to generate an interface.

B - Architecture

The visualization follow the next architecture :

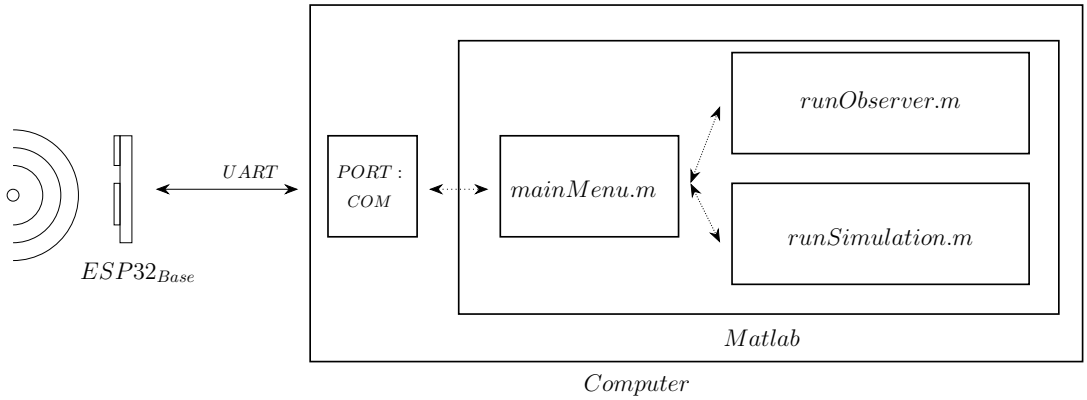


Figure I.1: Visualization architecture

Let's explain each different bloc and its role :

- ESP32 : Relay between ESP-NOW protocol and the UART Port of the computer, which permit to make it visible for the matlab code.
- COM PORT: Physical bridge between the computer and the ESP32
- mainMenu : General window which redirect for the next window
- runObserver : Window to observe UART communication on the PORT (Debug utility)
- runSimulation : Main Window where we visualise attitude of the drone, shape placement and data reader

5 - Simulation

On this project, outdoor tests are very rare due to logistical problem. However, we need to verify our work such as the corrector. To do so, we developed a simulation protocol using the real MAVLink messages, and same data as our drone.
In this part, we will first present the main simulation tools, ArduPilot and Mission planer, second, explain the implementation, then present the result we obtained and finally the simulation possibilities.

A - ArduPilot and Mission Planer

To do drone simulation, we have to select the same architecture as our drone on the pixhwak 4. In the drone industry, two differents architectures are existing: Ardupilot and PX4. In our case, pixhwak 4 is using the ArduPilot architectures.
Then, after selecting the architecture, we have to use a software to visualise and simulate drone. We choose mission planer because we have already work with it on different project.
Let's present both tools:

ArduPilot

ArduPilot was created in 2007 by the DIY Drones community. Originally developed for Arduino boards, it has since become a general-purpose, open-source autopilot. It can control many types of vehicles (copters, planes, rovers, submarines, etc.). ArduPilot provides autopilot functions, receives data from sensors, and fuses it using an Extended Kalman Filter (EKF) [ref]. It also allows users to create pre-planned missions. To operate, it uses the MAVLink protocol to communicate with the drone and facilitate control. In our case, an ESP32 uses the MAVLink library to send MAVLink messages that command the drone's movements.

Mission planer

Mission planer is a representative tool for copters such as drone. Created by Michael Oborne to help users of ArduPilot architecture to vizualise their drone and command them. The software also permits to declare multiple drone so create a swarm. For our project, it will be used as a simulation field.

B - Architecture

The simulation works with a very specific architecture cause tools like ArduPilot works under Linux system. The architecture is exposed in the figure below.

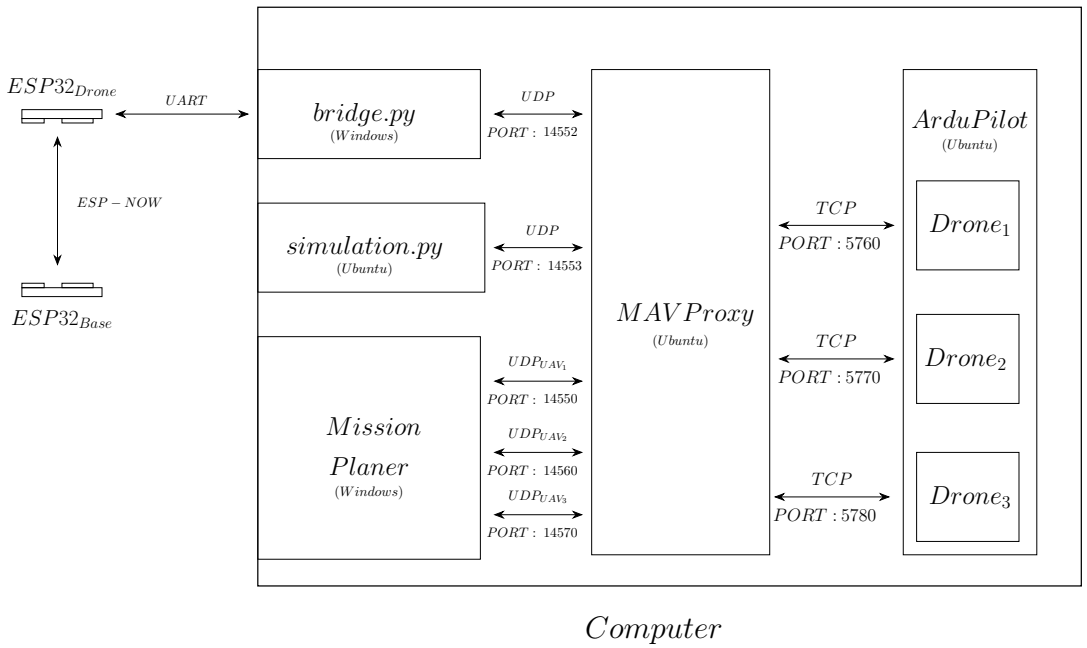


Figure I.2: Simulation architecture

Let's explain each different bloc and its role :

- $ESP - 32_{Base}$: Represent order send from the base for exemple takeoff
- $ESP - 32_{Drone}$: Represent the main drone where code we want to test is running
- ArduPilot : Represent the pixhawk of our drone. Here, we are simulating 3 of them.
- Bridge.py : Code made to redirect data from the UART connection to UDP connection
- MAVProxy : Manage all entries and exit permit to link the drone entries redirect them to the Ardupilot and also permit to redirect exit from ArduPilot (GPS, speed, etc) to Mission Planer.
- Mission Planer : Use to visualise our swarm drone based on data from ArduPilot
- Simulation.py : Python scrip to simulate $Drone_2$ and $Drone_3$

C - Result

Add after fist Test

D - Simulation possibilities

Add after fist Test

6 - Outdoor Test